

CHAPTER 13

Integrating Terraform with CI/CD Pipelines

Automated, consistent, and reliable infrastructure delivery with GitLab CI/CD and GitOps workflows.



GitLab CI/CD



GitOps Workflows



Automation

What we'll instruct today



13.1

Using Terraform with GitLab CI/CD

Repo setup, pipeline stages, remote state, secrets



13.2

Terraform GitOps Workflow

Merge-request driven plan & apply, formatting & linting



13.3

Automating Infrastructure Deployment

Multi-environment delivery, protecting production



13.4

Scenario & Chapter Summary

End-to-end walkthrough and key takeaways

SECTION 13.1

Using Terraform with GitLab CI/CD

Four steps to a working pipeline: repository, CI/CD definition, secure state, and protected secrets.

13.1

Create a GitLab Repository for Terraform

What to do

- ✓ Initialize a new Git repository
- ✓ Add Terraform config files (main.tf, variables.tf, etc.)
- ✓ Commit and push to your GitLab remote

Instructor tip: keep environments (dev/prod) in separate directories or workspaces from the start — it pays off later.

```
$ git init
$ git add .
$ git commit -m "Initial commit"
$ git remote add origin <your-gitlab-repo>
$ git push -u origin main
```

13.1

Define the .gitlab-ci.yml Pipeline

Three stages carry every change from code to infrastructure:



Manual gate on Apply prevents accidental changes from reaching real infrastructure.

13.1

Anatomy of the Pipeline Definition

```
stages:
  - validate
  - plan
  - apply

terraform_plan:
  stage: plan
  script:
    - terraform init
    - terraform plan -out=tfplan
  artifacts:
    paths:
      - tfplan
```



artifacts.paths

Passes the tfplan file from the plan job to the apply job.



stage ordering

GitLab runs validate → plan → apply in sequence automatically.

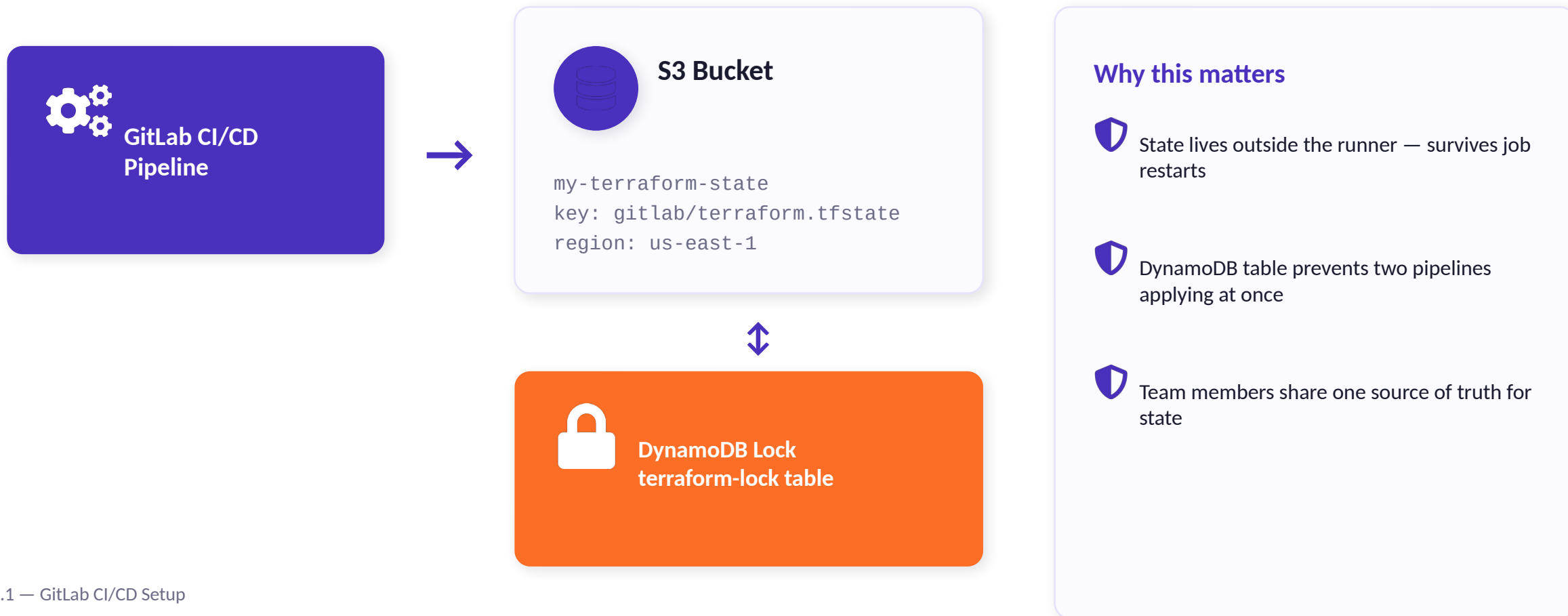


before_script

Installs the pinned Terraform binary once for every job.

13.1

Secure Terraform State with an S3 Backend



STEP 4

13.1

Store AWS Credentials as CI/CD Variables

Settings → CI/CD → Variables (never hardcode secrets in .gitlab-ci.yml)

Key	Value	Type
AWS_ACCESS_KEY_ID	AKIAEXAMPLE	Masked
AWS_SECRET_ACCESS_KEY	SECRET12345	Masked

```
variables:
```

```
  AWS_ACCESS_KEY_ID: $AWS_ACCESS_KEY_ID
```

```
  AWS_SECRET_ACCESS_KEY: $AWS_SECRET_ACCESS_KEY
```



Masked & protected variables never appear in job logs — keeping credentials out of pipeline output entirely.

SECTION 13.2

Terraform GitOps Workflow

A declarative approach where every infrastructure change is proposed, reviewed, and merged through Git.

Pull-Request (Merge Request) Workflow



13.2

Enforce Formatting & Linting

```
terraform_lint:  
  stage: validate  
  script:  
    - terraform fmt -check  
    - terraform validate
```



Blocks misformatted configuration



Rejects invalid configuration before it merges

Run this in the same “validate” stage as your plan job — catching style issues before they reach review saves reviewer time.

SECTION 13.3

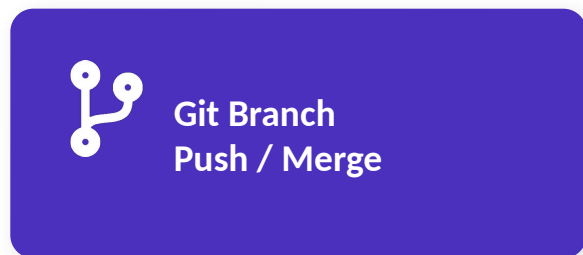
Automating Infrastructure Deployment

Route Terraform changes to the right environment automatically, based on branch.

STEP 1

13.3

Define Multiple Environments



dev branch
deploy_dev

```
environment: dev  
terraform apply -auto-approve
```



main branch
deploy_prod

```
environment: production  
terraform apply -auto-approve
```

Result

- dev branch changes deploy to the development environment
- main branch changes deploy to production
- Each stage uses GitLab's environment tracking for visibility

Protect the main Branch



Require approvals

No apply reaches production without a reviewed, approved Merge Request.



MRs for every update

All infrastructure changes flow through Merge Requests — no direct pushes.



Prevents accidental changes

Production stays isolated from unreviewed or experimental work.

HANDS-ON WALKTHROUGH

Set Up a GitLab CI/CD Pipeline for Terraform

Problem: You want to automate infrastructure deployment using GitLab CI/CD and Terraform.

- 1 Create a GitLab repository for Terraform code
- 2 Define a .gitlab-ci.yml pipeline
- 3 Store secrets in GitLab CI/CD Variables
- 4 Use GitOps workflows with Merge Requests

Expected Outcome

- ✓ Terraform runs on every commit
- ✓ Infrastructure is version-controlled
- ✓ Secrets are securely stored in GitLab

CHAPTER 13 SUMMARY

Key Takeaways



GitLab CI/CD pipeline

Validate, plan, and apply Terraform automatically on every commit.



GitOps workflows

Manage infrastructure changes through reviewed Merge Requests.



Secure secrets management

Store credentials safely in GitLab CI/CD Variables.



Multi-environment automation

Deploy to dev and production based on branch, with production protected.